Expression Language (EL) in JSP

Kishan

Expression Language (EL) is a powerful feature added to JSP to make it easier to access and manipulate application objects like request, response, out, etc.Known as JSP-EL, it simplifies JSP programming and improves readability.

The syntax for EL expressions: **${expression}**

There are ***11*** implicit Objects available in Expression Language which are listed below

Kishan

applicationScope : Refers to the ServletContext object. Used to access attributes stored in the application scope.
Example: ${applicationScope.attributeName}

sessionScope : Refers to the HttpSession object. Used to access attributes stored in the session scope
Example: ${sessionScope.attributeName}

requestScope : Refers to the HttpServletRequest object. Used to access attributes stored in the request scope

Example: ${requestScope.attributeName}

pageScope : Refers to the PageContext object. Used to access attributes stored in the page scope.

Example: ${pageScope.attributeName}

param : Used to retrieve a single parameter value from the request object.

Example: ${param.parameterName}

Kislan

paramValues : Used to retrieve multiple parameter values as a String array from the request object.

Example: ${paramValues.parameterName[0]}

header: Used to retrieve a single header value from the request object.

Example: ${header.headerName}

headerValues : Used to retrieve multiple header values as a String array from the request object.

Example: ${headerValues.headerName[0]}

cookie : Used to access cookies from the request object.

Example: ${cookie.cookieName.value}

initParam : Refers to the initialization parameters from the ServletConfig object.

Example: ${initParam.parameterName}

pageContext : Refers to the PageContext object. Provides access to all JSP objects.

Example: ${pageContext.request.method}

Kishan

JSTL (JSP Standard Tag Library)

Kishan

JSTL is a set of predefined tags to simplify JSP development. It removes the need for scriptlets by providing tag-based approach.

Advantages of JSTL:

- Fast Development
- Code Reusability
- No need to use scriptlet tag.

Kishan

In JSTL there are‘5’ tags, which are given below:

1. Core Tags
2. Function Tags
3. Formatting Tags
4. XML Tags
5. SQL Tags

Note:

To work with EL-JSTL, we need to download **jstl-1.2.jar** and copy that jar file into "lib" folder of WEB-INF. we need to use "**@taglib**" directive tag to declare JSTL Tags.

1) Core Tags: These JSTL core tags provides variable support, URL management, flow control etc. (Basic code writing)

syntax: <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>

The following are the List of JSTL Core Tags:

c:out =>It displays the result of an expression, similar to <%=...%> tag.

c:import =>It retrieves relative or an absolute URL.

c:set => It sets the value to variable.

c:remove =>It is used for removing the variable .

c:catch => It is used for Catching any Throwable exception that occurs in the body.

c:if => It is an conditional tag

Kishan

c:choose, c:when,c:otherwise => It is the simple conditional tag that includes its body content if the evaluated condition is true.

c:forEach => It is the basic iteration tag.

c:forTokens => It iterates over tokens which is separated by the supplied delimeters.

Kishan

c:param => It adds a parameter in a containing 'import' tag's URL.

c:redirect => It redirects the browser to a new URL and supports the context-relative URLs.

c:url => It creates a URL with optional query parameters.

2) Function Tags: These JSTL function tags provides a number of standard functions, most of these functions are common string manipulation functions.

syntax: <%@taglib uri= "http://java.sun.com/jsp/jstl/functions« prefix="fn" %>

The following are the List of some JSTL Function Tags:

fn:contains() :It is used to test if an input string containing the specified substring or not.

fn:containsIgnoreCase() : It is used to test if an input string contains the specified substring as a case insensitive way.

fn:endsWith() : It is used to test if an input string ends with the specified suffix.

fn:indexOf() : It returns an index within a string of first occurrence of a specified substring.

fn:trim() : It removes the blank spaces from both the ends of a string.

Kishan

fn:startsWith() : It is used for checking whether the given string is started with a particular string value or not.

fn:split() : It splits the string into an array of substrings.

fn:toLowerCase() : It converts all the characters of a string to lowercase.

fn:toUpperCase() : It converts all the characters of a string to uppercase.

fn:substring() : It returns the subset of a string according to the given start and end position.

fn:length() : It returns the number of characters inside a string, or the number of items in a collection.

fn:replace() : It replaces all the occurrence of a string with another string sequence.

Kishan

3)Formatting Tags: These formatting tags provide support for message formatting, number formating and date formatting etc. These formatting tags are also used for internationalized web sites to display and format text, time, date and numbers.

syntax: <%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>

The following are the List of some JSTL Function Tags:

fmt:parseNumber : It is used to Parse the string representation of a currency, percentage or number.

fmt:formatNumber : It is used to format the numerical value with specific format or precision.

fmt:parseDate : It parses the string representation of a time and date.

fmt:setTimeZone: It stores the time zone inside a time zone configuration variable.

fmt:formatDate : It formats the time and date using the supplied pattern and styles.

Kishan

4) XML Tags:

The JSTL XML tags are used for providing a JSP-centric way of manipulating and creating XML documents.

syntax:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/xml" prefix="x" %>
```

Kishan

5) SQL Tags:

The SQL tag library allows the tags to Interact with RDBMS (Relational Databases) such as Microsoft SQL Server, mySQL, or Oracle.

syntax:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql" %>
```

Web Application Architectures in Java

Kishan

Java web applications can be developed using two primary models

- Model-1 Architecture
- Model-2 Architecture

Model-1 Architecture:

In this architecture JSP (Java Server Pages) handles both the presentation logic and business logic. JSP is responsible for all functionalities, including receiving requests, processing data, and generating responses. Entire application logic resides in JSP pages.

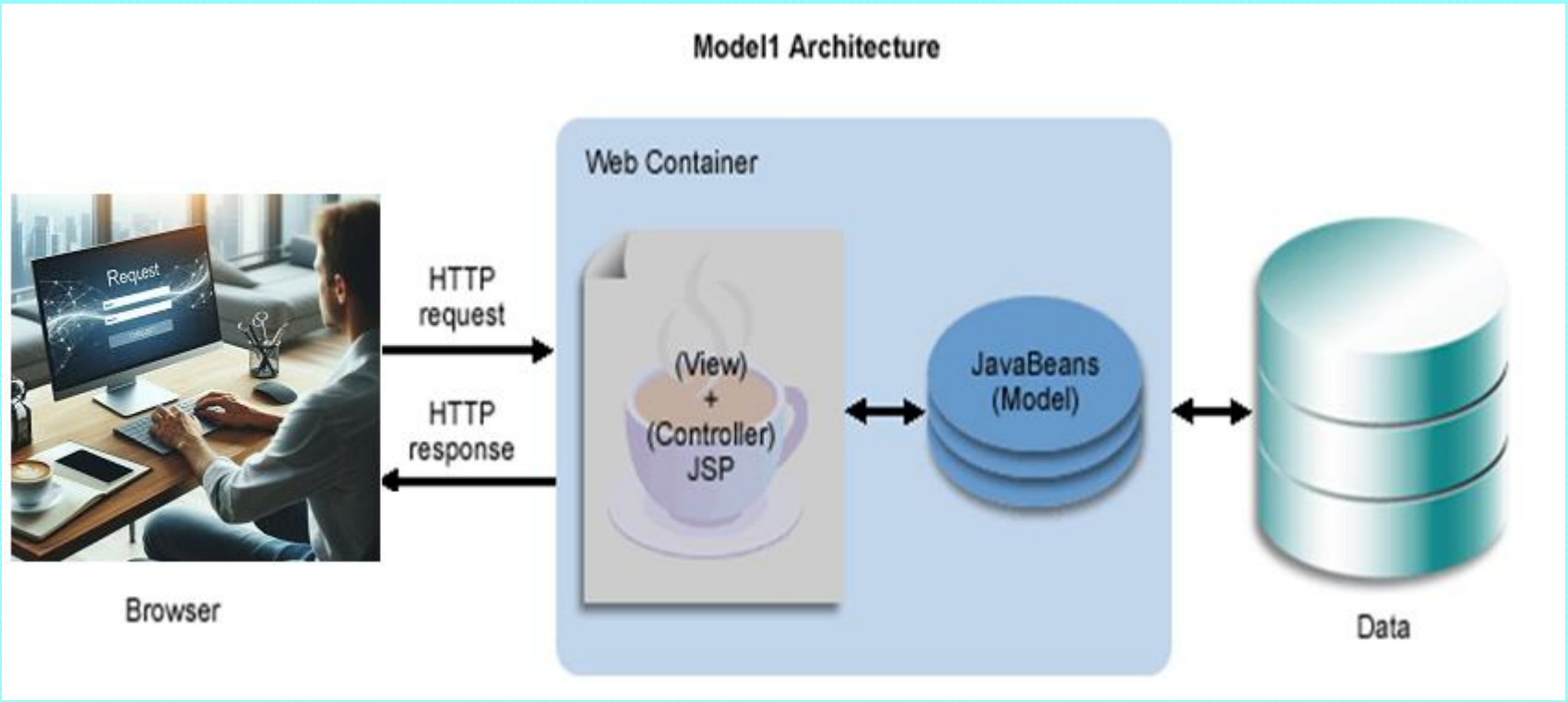
Limitations:

1) Inflexibility : Modifications in one JSP page can lead to unexpected changes in others.

2) Coupled Responsibilities :Developers must handle both presentation and business logic in a single layer.

3) Scalability Issues : Larger applications lead to increasingly complex JSP pages.

Kishan


Model1 Architecture
Web Container
Request
HTTP
request
HTTP
response
(View)
+
(Controller)
JSP
JavaBeans
(Model)
Browser
Data

Model-2 Architecture:

An improved model based on the **MVC** (Model-View-Controller) design pattern, addressing the limitations of Model-1.

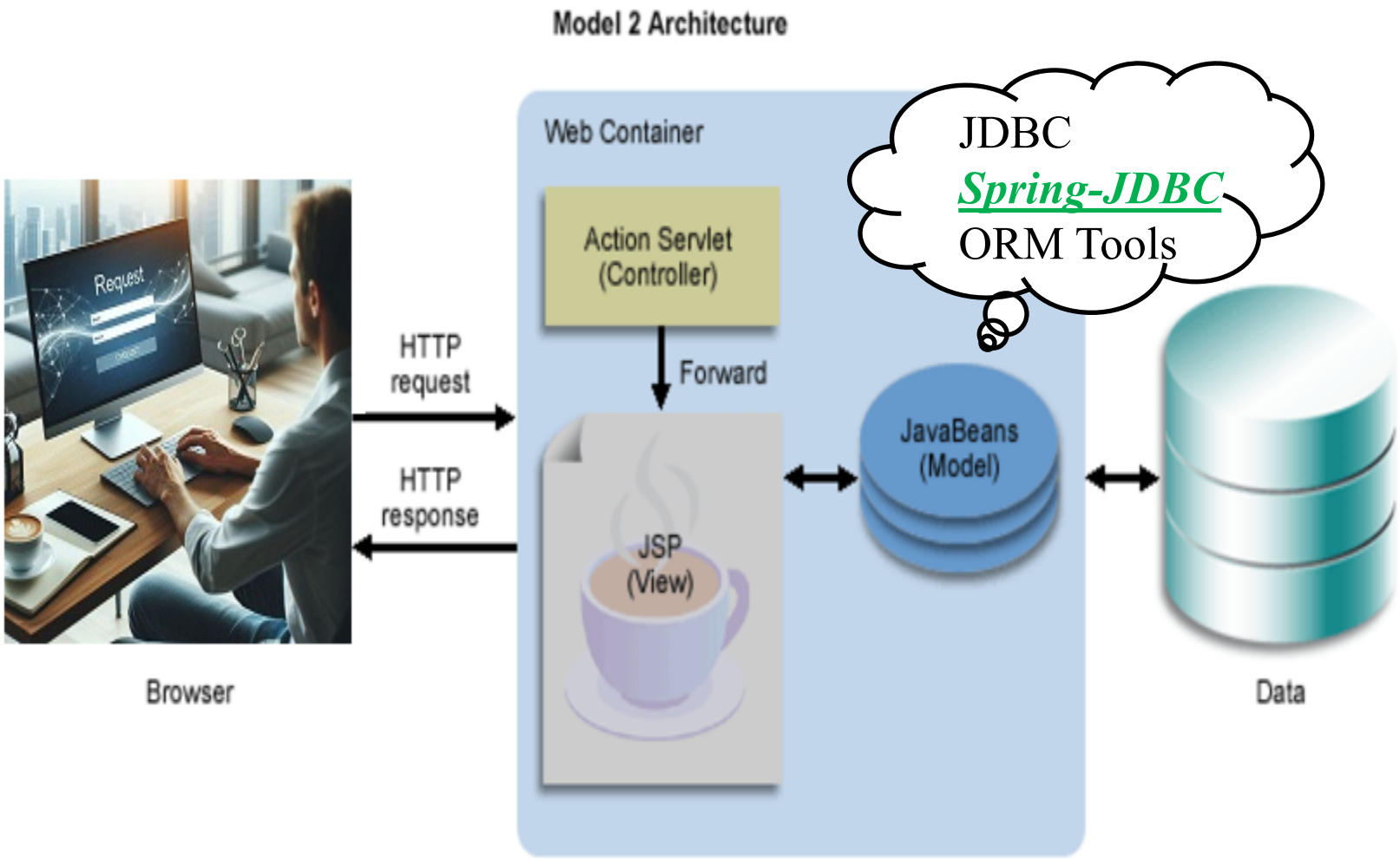
Components:

Model: Represents data and business logic. Typically implemented using JavaBeans or POJOs.

View: Handles presentation logic. Implemented using JSP, which fetches data from the Model to display.

Controller: Manages the application's flow. Handles client requests, invokes business logic, and decides which View to display next. Typically implemented using Servlets.

Kishan


Model 2 Architecture
Web Container
Action Servlet
(Controller)
Forward
JSP
(View)
JavaBeans
(Model)
HTTP
request
HTTP
response
Browser
Data
JDBC
Spring-JDBC
ORM Tools

Advantages:

- Separation of Code : Clean separation of business logic (**M**odel), user interface (**V**iew), and control flow (**C**ontroller).
- Reusability: Business logic components can be reused across applications.
- Flexibility: Presentation logic (View) can be modified independently of the business logic (Model).
- Scalability: Well-suited for large, complex web applications.

Kishan

Error Codes in Servlets

Kishan

Error Codes in Servlets:

An Error Code is a numeric representation of the type of error that occurs during the execution of a web application. These codes help in identifying the nature of errors and in handling them appropriately. Applications may use error codes with messages to indicate fatal issues that require termination. The HttpServletResponse interface provides predefined constants to represent common HTTP error codes and statuses. The ***sendError()*** method in HttpServletResponse is used to send an error response to the client

Kishan

Syntax :

```
public abstract void sendError(int statusCode, String message)
                                        throws java.io.IOException;
```

Code	Message	Description
100	Continue	Only a part of the request has been received by the server, but as long as it has not been rejected, the client should continue with the request
101	Switching Protocols	The server switches protocol.
200	OK	The request is OK
201	Created	The request is complete, and a new resource is created
202	Accepted	The request is accepted for processing, but the processing is not complete.
203	Non-authoritative Information	The server successfully processed the request, but the response payload is modified from the original by a transforming proxy or intermediary. It means the returned information comes from a source that is not the authoritative origin.
204	No Content	The server successfully processed the request but does not return any content. The client should not change its view (e.g., a browser will not refresh the page).
205	Reset Content	The server successfully processed the request, but it instructs the client to reset the view (e.g., reset form inputs or clear user interfaces).
206	Partial Content	The server successfully processed a partial GET request, typically to deliver a specific range of data (e.g., for file download resumption or streaming media).
300	Multiple Choices	A link list. The user can select a link and go to that location. Maximum five addresses
301	Moved Permanently	The requested page has moved to a new url

Code	Message	Description
302	Found	The requested page has moved temporarily to a new url
303	See Other	The requested page can be found under a different url
304	Not Modified	Indicates that the requested resource has not been modified since the last request. The server does not send the resource again but allows the client to use its cached version.
305	Use Proxy	Indicates that the requested resource must be accessed through a proxy specified in the response's Location header.
306	Unused	This code was used in a previous version. It is no longer used, but the code is reserved
307	Temporary Redirect	The requested page has moved temporarily to a new url.
400	Bad Request	The server did not understand the request
401	Unauthorized	The requested page needs a username and a password
402	Payment Required	You cannot use this code yet
403	Forbidden	Access is forbidden to the requested page
404	Not Found	The server cannot find the requested page.
405	Method Not Allowed	The method specified in the request is not allowed.

Code	Message	Description
406	Not Acceptable	The server can only generate a response that is not accepted by the client.
407	Proxy Authentication Required	You must authenticate with a proxy server before this request can be served.
408	Request Timeout	The request took longer than the server was prepared to wait.
409	Conflict	The request could not be completed because of a conflict.
410	Gone	The requested page is no longer available.
411	Length Required	The "Content-Length" is not defined. The server will not accept the request without it.
412	Precondition Failed	The precondition given in the request evaluated to false by the server.
413	Request Entity Too Large	The server will not accept the request, because the request entity is too large.
414	Request-url Too Long	The server will not accept the request, because the url is too long. Occurs when you convert a "post" request to a "get" request with a long query information.
415	Unsupported Media Type	The server will not accept the request, because the media type is not supported.
417	Expectation Failed	

Code	Message	Description
500	Internal Server Error	The request was not completed. The server met an unexpected condition.
501	Not Implemented	The request was not completed. The server did not support the functionality required.
502	Bad Gateway	The request was not completed. The server received an invalid response from the upstream server.
503	Service Unavailable	The request was not completed. The server is temporarily overloading or down.
504	Gateway Timeout	The gateway has timed out.
505	HTTP Version Not Supported	The server does not support the "http protocol" version.